CSE235 Week 10: Classes

Classes allow data and functionality to be associated with a single variable. A variable with a class type is called an object. We've been using objects throughout the course; strings, lists, file objects, and dictionaries are all classes that we create objects of. The data and functions associated with an object are accessed via the dot operator. Any of the functions that we've called like `a.b()` are objects.

Defining a Class

Classes in Python are defined using the `class` keyword, followed by a name for the class and colon. The body of a class is indented. The body of a class holds variables and functions:

```
class Example:
    var = 0

    def function():
        pass
```

Python member functions require a first parameter called `self` (technically functions don't need this, but then they cannot be called by objects using the dot operator, and aren't particularly useful). This parameter acts as reference to the object that is calling the function. Before looking at general member functions, let's look at "magic methods". Magic methods are special functions that classes have. They are automatically called in certain situations. All the magic methods start and end with two underscores. The first one we'll discuss is `__init__`. If you are familiar with other programming languages, `__init__()` is Python's version of a constructor. If you aren't familiar with constructors, they are special functions that create new objects. Constructors set the member variables (also called fields) for the object. Because Python does not have declarations in the same way that other languages do, the fields of the object are set in the `__init__()` function. While not a technical requirement, `__init__()` should

always be passed the `self` parameter, so it can set the state of the object being created.

```
class Student:
    def __init__(self, name, gpa, credits):
        # create the properties using self
        self.name = name
        self.gpa = gpa
        self.credits = credits


# call the constructor using the class name
john_doe = Student("John Doe", 3.5, 54)
# we can access the fields of the object
print(f"Name: {john_doe.name}")
# prints: Name: John Doe

# create another object
jane_doe = Student("Jane Doe", 3.6, 27)
```

The `__str__()` function returns an "informal" string representation of the object. This is used by `print()` when outputting an object, and is the "pretty print" version of the object:

```
class Student:
    def __init__(self, name, gpa, credits):
        # create the properties using self
        self.name = name
        self.gpa = gpa
        self.credits = credits


    def __str__(self):
        s = f"{self.name} has completed "
        s += f"{self.credits} credit hours "
        s += f"with a GPA of {self.gpa}"
        return s


john_doe = Student("John Doe", 3.5, 54)
print(john_doe)
# prints: John Doe has completed 54 credit hours with a GPA of 3.5
jane_doe = Student("Jane Doe", 3.6, 27)
print(jane_doe)
# prints: Jane Doe has completed 27 credit hours with a GPA of 3.6
```

The `__repr__()` is the "official" string representation of an object. It is

used by the `repr()` function, and is used for debugging and printing during exception handling. This should return a string that is as descriptive as possible about the state of the object. If `__str__()` is not defined, `__repr__()` will be called instead.

```
class Student:
    def __init__(self, name, gpa, credits):
        # create the properties using self
        self.name = name
        self.gpa = gpa
        self.credits = credits


    def __str__(self):
        s = f"{self.name} has completed "
        s += f"{self.credits} credit hours "
        s += f"with a GPA of {self.gpa}"
        return s


    def __repr__(self):
        s = "class Student:\n"
        s += f"name: '{self.name}'\n"
        s += f"gpa: {self.gpa}\n"
        s += f"credits: {self.credits}"
        return s


john_doe = Student("John Doe", 3.5, 54)
print(john_doe.__repr__())
# prints:
# class Student:
# name: 'John Doe'
# gpa: 3.5
# credit_hours: 54
jane_doe = Student("Jane Doe", 3.6, 27)
print(jane_doe.__repr__())
# prints:
# class Student:
# name: 'Jane Doe'
# gpa: 3.6
# credit_hours: 27
```

The `__eq__()` magic method is used to compare if two objects are equal. It is called when the `==` operator is used. If this function is not defined, then `==` cannot be used. `__lt__()` is used to implement `<`, `__gt__()` is used to implement `>`. `__ne__()` corresponds to `!=`,

`__le__()` is `<=`, and `__ge__()` is `>=`. The `isinstance()` function is used to determine if a variable has a certain type. While `self` will be an object of the class, the second parameter may not be. The `isinstance()` function is used to check the type of a variable or parameter. If the type does not match, we can return `False` or `NotImplemented`.

```
class Student:
    def __init__(self, name, gpa, credits):
        # create the properties using self
        self.name = name
        self.gpa = gpa
        self.credits = credits


    def __eq__(self, rhs):
        if not isinstance(rhs, BankAccount):
            return NotImplemented
        # unfortunately this gets cut off when converting to a PDF
        # the last and is "self.credits == rhs.credits"
        if self.name == rhs.name and self.gpa == rhs.gpa and self.cred
            return true
        else:
            return false


    def __lt__(self, rhs):
        if not isinstance(rhs, BankAccount):
            return NotImplemented
        if self.name < rhs.name:
            return true
        else:
            return false


    def __gt__(self, rhs):
        if not isinstance(rhs, BankAccount):
            return NotImplemented
        return lhs < self and not (self == rhs)


    def __ne__(self, rhs):
        if not isinstance(rhs, BankAccount):
            return NotImplemented
        return not (self == rhs)


    def __le__(self, rhs):
        if not isinstance(rhs, BankAccount):
```

```
            return NotImplemented
        return (self < rhs) and (self == rhs)


    def __ge__(self, rhs):
        if not isinstance(rhs, BankAccount):
            return NotImplemented
        return rhs < self and self == rhs
```

The `__hash__()` magic method is called by the `hash()` function, such as when the object is being inserted into a set or used as a key to a dictionary. A basic way to implement `__hash__()` is to call the `hash()` function for all the fields of the class:

```
class Student:
    def __init__(self, name, gpa, credits):
        # create the properties using self
        self.name = name
        self.gpa = gpa
        self.credits = credits


    def __hash__(self):
        return hash(self.name, self.gpa, self.credits)
```

Let's look at another example. Classes are used to represent domain entities. Here is an example class that represents a bank account. We also define some non-magic methods called `deposit()` and `withdraw()`. These member functions still need to be called by individual objects, so they need the `self` parameter.

```
class BankAccount:
    def __init__(self, holder, account_number, initial_balance):
        self.holder = holder
        self.number = account_number
        self.balance = initial_balance


    def __eq__(self, rhs):
        holder_equal = self.holder == rhs.holder
        number_equal = self.number == rhs.number
        balance_equal = self.balance == rhs.balance
        return holder_equal and number_equal and balance_equal


    def __repr__(self):
```

```
        s = "class BankAccount\n"
        s += f"holder: {self.holder}\n"
        s += f"number: {self.number}\n"
        s += f"balance: {self.balance}"
        return s


    def __str__(self):
        return f"\n{self.number} | {self.holder} | ${self.balance:.2f}"


    # normal member functions have the self parameter as well
    def deposit(self, amount):
        if amount < 0:
            raise "Cannot deposit negative amount"
        else:
            self.balance += amount


    def withdraw(self, amount):
        if amount < 0:
            raise "Cannot withdraw negative amount"
        if self.balance < amount:
            raise "Overdraft prevented"
        else:
            self.balance -= amount
```

Inheritance

Classes in Python can inherit from another class. The class being inherited from is called the base class, super class or parent class. The class doing the inheriting is called the derived class, subclass or child class. A derived class has access to all of it's base class's data members and functions. The `super()` function allows the super class's methods to be called directly. Base class methods are Let's create two subclasses of the earlier `BankAccount` example, `CheckingAccount` and `SavingsAccount`:

```
# the base class goes in parentheses after the class name
class CheckingAccount(BankAccount):
    def __init__(self, holder, account_number, initial_balance):
        # call the super class's constructor
        super().__init__(holder, account_number, initial_balance)


    # override the behavior of withdraw
    def withdraw(self, amount):
```

```
        if amount < 0:
            raise "Cannot withdraw negative amount"
        if self.balance < amount:
            # instead of preventing overdraft, apply a fee
            self.balance -= amount
            self.balance -= 50
        else:
            self.balance -= amount

    # the rest of the member functions are inherited from BankAccount


class SavingsAccount(BankAccount):
    # add additional parameter for interest rate
    def __init__(self, holder, account_number, initial_balance, rate):
        self.rate = rate
        # use base constructor for the rest
        super().__init__(holder, account_number, initial_balance)


    # override __repr__ and __str__
    def __repr__(self):
        s = super().__repr__()
        s += f"rate: {self.rate}"
        return s


    def __str__(self):
        s = super().__str__()
        s += f" | {self.rate}"
        return s


    # add a new member function to add interest
    def add_interest(self):
        # don't need to call super() to access fields
        self.balance = self.balance + (self.balance * self.rate)


savings_account = SavingsAccount("John Doe", 1000, 1000, 0.025)
# we can still call the base class members
savings_account.deposit(200)
print(savings_account)
# prints: John Doe | 1000 | $1000.00 | 0.025

# we can also call the subclass's specific function
savings_account.add_interest()
print(savings_account)
# prints: 1000 | John Doe | $1230.00 | 0.025

# declare a base class object
```

```
bank_account = BankAccount("Jane Doe", 1001, 1000)
print(bank_account)
# prints: 1001 | Jane Doe | $1000.00
# we can't call add_interest() on bank_account
bank_account.add_interest()
```

Output:

```
1000 | John Doe | $1200.00 | 0.025
1000 | John Doe | $1230.00 | 0.025
1001 | Jane Doe | $1000.00
Traceback (most recent call last):
  File "bank_account_example.py", line 105, in <module>
    bank_account.add_interest()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'BankAccount' object has no attribute 'add_interest'
```

Conclusion

This week, we discussed classes. Classes provide a mechanism to create new data types that are combinations of other data types. Classes also allow you to define functions that operate on the data of the class. The term object is used to refer to a class variable. Each object has its own data, which makes the separate objects of a class unique. Python has multiple "magic methods" which are automatically called in certain situations. Implementing these methods allows objects of a class to respond to these situations. The `self` parameter allows member functions in a class to access the individual data members of the specific object calling the function.
